

Face Morphing - Final Project Report

1. Method

We implement classical landmark-based face morphing: given two face images of different identities, we (a) detect dense facial landmarks on each face, (b) establish point correspondence via matching landmark indices, (c) triangulate the averaged landmark shape with Delaunay triangulation, (d) warp each source image's triangles into the corresponding triangles of the target (morph) shape with a piecewise-affine transform, and (e) cross-dissolve (linearly blend) the two warped images. This is the same family of technique used to generate face-morphing-attack images in the biometrics literature (e.g. Ferrara, Franco & Maltoni, "The Magic Passport," IJCB 2014).

2. Data

Five or more pairs of different-identity face images are drawn from Labeled Faces in the Wild (LFW) via scikit-learn's `fetch_lfw_pairs` "different person" split -- a standard, public, pre-aligned face dataset, which avoids privacy/consent concerns around using photos of people we know while still giving realistic, varied bona fide face images.

3. Landmark detection & alignment strategy

Landmarks are obtained with MediaPipe's FaceLandmarker (a 478-point 3-D face mesh; LFW crops are 125x94 px and are bicubically upsampled 3x first for reliable detection). Because MediaPipe only tracks the face region, 8 fixed boundary points (image corners + edge midpoints) are appended to both landmark sets so the background also participates in the warp instead of only the face.

Correspondence between the two faces is purely index-based: landmark i on face A corresponds to landmark i on face B, since both come from the same 478-point model with a fixed topology -- no separate registration/ICP step is needed. The morph target shape at blend factor α is the per-point convex combination $(1-\alpha)\text{ptsA} + \alpha\text{ptsB}$, and the Delaunay triangulation used for warping is computed on this averaged shape (not on either source shape individually), so one triangle topology is simultaneously valid for warping both source images.

4. Interpolation technique

For a chosen α in $[0,1]$, each of the ~ 950 Delaunay triangles is affine-warped from face A, and separately from face B, into its position in the α -blended target shape (`cv2.getAffineTransform + warpAffine`); the two warped triangle patches are then cross-dissolved with weights $(1-\alpha, \alpha)$ and composited using a filled-polygon mask. This reproduces both the geometric interpolation (the shape warps smoothly with α) and the photometric interpolation (pixel intensities blend smoothly with α) that a real morphing attack relies on. We render the standard attack morph at $\alpha=0.5$ for every pair, plus an $\alpha = 0, 0.25, 0.5, 0.75, 1$ strip for a subset, to visualize the interpolation continuum (Figure 1).



Figure 1: interpolation strip at $\alpha = 0, 0.25, 0.5, 0.75, 1$.

5. Observations & limitations

- At $\alpha=0.5$ the morphs are visually convincing single identities that plausibly resemble both source subjects -- exactly the property that makes morphing attacks a biometric security threat.
- Quality depends heavily on how well the two source photos agree in pose and expression: pairs with mismatched mouth state (one subject smiling with teeth, the other neutral) show visible ghosting around the mouth.
- A feature present in only one subject (e.g. eyeglasses) fades in/out across α rather than looking physically consistent -- a classic, visible morphing artifact.
- Small seams appear near the image border, since background pixels far from any landmark are only weakly constrained by the triangulation.
- We use simple cross-dissolve rather than Poisson/gradient-domain blending, so skin-tone/lighting differences between the two subjects are not colour-corrected -- a known limitation relative to production-grade morphing tools.
- Source image resolution (125x94 before upscaling) caps the fine detail achievable in the final morph.



Figure 2: subject A, morph ($\alpha=0.5$), subject B -- one of the five showcased pairs (see [outputs/showcase/](#) for all five, and [outputs/pairs/](#) + [outputs/morphs/](#) for every generated pair).